

Evolution of Reinforcement Learning

From Dynamic Programming to RLHF and Reasoning Models

Ye Kyaw Thu, Language Understanding Lab., Myanmar

AI Engineering (Fundamental) Class

June 21, 2026

Lecture Roadmap (2 Hours)

1. The Foundation: MDPs and Returns
2. Dynamic Programming (DP)
3. Monte Carlo (MC)
4. Temporal Difference Learning (TD)
5. Model-Based RL: Dyna-Q
6. Policy Gradients (REINFORCE)
7. Actor-Critic (A2C)
8. Deep Q-Networks (DQN)

The Reinforcement Learning Loop

Reinforcement Learning (RL) is about an **Agent** learning by interacting with an **Environment**.

- **State (S_t):** The current situation (e.g., coordinates on a grid).
- **Action (A_t):** What the agent decides to do (e.g., move Up, Down, Left, Right).
- **Reward (R_{t+1}):** The score received for taking that action (e.g., +1 for goal, -1 for trap).

Key Insight (Sutton & Barto, Ch. 1)

RL differs from supervised learning: there are no labeled examples. The agent must discover which actions yield the most reward by trying them.

The Goal: Maximizing Return (G_t)

The agent doesn't just want immediate reward; it wants the maximum total reward over time. We call this the **Return**.

Discounted Return Equation

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Symbol Breakdown: Return (G_t)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- G_t : The total Return starting from time step t .
- R_{t+1} : The immediate reward received for the first action.
- γ (Gamma): The **Discount Factor**. A number between 0 and 1.
 - If $\gamma = 0$, the agent only cares about immediate reward.
 - If $\gamma = 0.9$, the agent cares about the future, but future rewards lose 10% of their value each step.

Toy Calculation: Discounted Return

Scenario: An agent takes 3 steps. It gets a reward of 0, then 0, then +1 at the goal. The discount factor $\gamma = 0.9$.

Calculation:

$$G_0 = R_1 + (\gamma \times R_2) + (\gamma^2 \times R_3)$$

$$G_0 = 0 + (0.9 \times 0) + (0.9^2 \times 1)$$

$$G_0 = 0 + 0 + (0.81 \times 1)$$

$$G_0 = 0.81$$

Meaning: Even though the goal gives +1, from the starting position 3 steps away, the *value* of that path is 0.81.

Markov Decision Processes (MDPs)

An MDP formally defines the RL problem (Sutton & Barto, Ch. 3):

- **State Space** $\mathcal{S}$: All possible situations
- **Action Space** $\mathcal{A}$: All possible actions
- **Transition Dynamics** $P(s'|s, a)$: Probability of reaching s' from state s taking action a
- **Reward Function** $R(s, a, s')$: Immediate reward
- **Discount Factor** $\gamma \in [0, 1]$

The Markov Property

The future is independent of the past given the present: $P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1}|S_t, A_t)$

Dynamic Programming: The Perfect Map

Dynamic Programming requires a **perfect model** of the environment. We must know the exact probability of every transition.

Bellman Optimality Equation

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

We calculate the Value of a state ($V(s)$) by looking at all possible next states (s'), multiplying their values by the probability of getting there (P).

Model-Based Method: Requires complete knowledge of $P(s'|s, a)$

Symbol Breakdown: Bellman Equation

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

- $V^*(s)$: The optimal value of the current state.
- $\max_a$: Look at all actions and choose the best one.
- $\sum_{s'}$: Add up the math for all possible next states we might land in.
- $P(s'|s, a)$: The probability of landing in state s' if we take action a .
- $R(s, a, s')$: The reward we get for making that specific jump.
- $V^*(s')$: The value of the *next* state.

Toy Calculation: Dynamic Programming

Scenario: From state A, taking action "Move Right". There is an 80% chance we hit the Goal (Reward +1, Next Value 0). There is a 20% chance we slip into a Trap (Reward -1, Next Value 0). $\gamma = 0.9$.

Calculation for "Move Right":

$$Q(A, \text{Right}) = [0.8 \times (1 + 0.9 \times 0)] + [0.2 \times (-1 + 0.9 \times 0)]$$

$$Q(A, \text{Right}) = [0.8 \times 1] + [0.2 \times -1]$$

$$Q(A, \text{Right}) = 0.8 - 0.2$$

$$Q(A, \text{Right}) = 0.6$$

Result in Code (dp.py): This is exact mathematical planning without ever playing the game.

DP: Policy Iteration vs Value Iteration

Two classic DP algorithms (Sutton & Barto, Ch. 4):

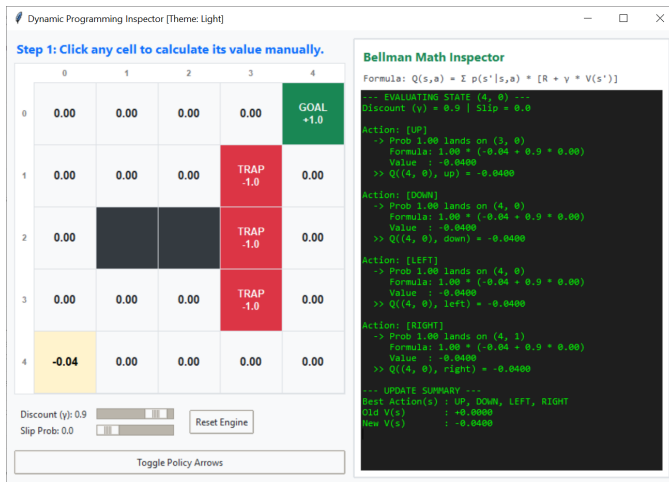
Policy Iteration:

- 1 **Policy Evaluation:** Given policy π , compute $V^\pi(s)$
- 2 **Policy Improvement:** Greedily improve π using V^π
- 3 Repeat until convergence

Value Iteration:

- 1 Apply Bellman optimality backup repeatedly
- 2 $V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a)[R + \gamma V(s')]$
- 3 Extract optimal policy from V^*

DP Visualization



Click any cell to see the Bellman equation calculation in real-time. The heatmap shows state values; arrows show the optimal policy.

DP: Pros and Cons

Pros:

- Guarantees optimal solution
- Mathematically rigorous
- Provides exact value functions
- Foundation for understanding all RL

Cons:

- Requires **complete model** (P, R)
- Computationally expensive: $O(|\mathcal{S}|^2|\mathcal{A}|)$
- Curse of dimensionality
- Impractical for large/continuous state spaces

When to Use DP

Small, fully observable environments where the model is known (e.g., board games with known rules, simple robotics).

Monte Carlo: Learning from Experience

What if we don't know the probabilities of the environment? We cannot do the math in advance. We must learn by **trial and error**.

Monte Carlo Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

Monte Carlo methods play an entire episode to the end, look at the final Return (G_t), and then update the values of the states they visited.

Model-Free Method: No knowledge of $P(s'|s, a)$ required

Symbol Breakdown: Monte Carlo

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

- $V(S_t)$: Our current guess for the value of the state.
- $\leftarrow$: "Update this variable to become..."
- α (Alpha): The **Learning Rate**. How big of a step we take toward the new information (e.g., 0.1).
- G_t : The actual Return we got after finishing the episode.
- $[G_t - V(S_t)]$: The **Error**. The difference between what actually happened (G_t) and what we guessed would happen ($V(S_t)$).

Toy Calculation: Monte Carlo

Scenario: Our agent is in state A. It currently guesses $V(A) = 0.0$. It plays an episode, reaches the goal, and calculates the actual return was $G_t = 1.0$. Our learning rate $\alpha = 0.1$.

Calculation:

$$V(A) \leftarrow 0.0 + 0.1 \times [1.0 - 0.0]$$

$$V(A) \leftarrow 0.0 + 0.1 \times [1.0]$$

$$V(A) \leftarrow 0.1$$

Meaning: The agent's guess for state A improved from 0.0 to 0.1. If it plays 100 times and keeps getting +1.0, $V(A)$ will slowly rise until it reaches 1.0. (mc.py)

MC: First-Visit vs Every-Visit

Two variants (Sutton & Barto, Ch. 5):

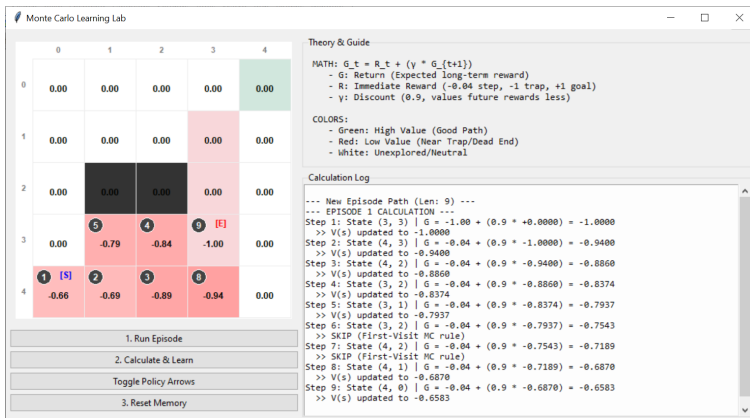
- **First-Visit MC:** Only update $V(S_t)$ the *first time* state S_t is visited in an episode
- **Every-Visit MC:** Update $V(S_t)$ *every time* state S_t is visited

First-Visit MC Update

$$V(S_t) \leftarrow \text{average}(G_t \text{ for all episodes where } S_t \text{ was first visited})$$

First-Visit MC has less bias and is more commonly used in practice.

MC Visualization



Run episodes to see values update. Green cells indicate high-value states leading to the goal; red indicates trap proximity.

MC: Pros and Cons

Pros:

- Model-free: no $P(s'|s, a)$ needed
- Unbiased estimates of $V(s)$
- Simple to understand and implement
- Can learn from complete episodes only

Cons:

- High variance in returns
- Must wait for episode to end
- Inefficient for long episodes
- Cannot learn from incomplete episodes

When to Use MC

Episodic tasks with clear termination (e.g., games, card playing, episodic control).

Temporal Difference: Step-by-Step Learning

Monte Carlo waits until the end of the episode to learn. **Temporal Difference (TD)** learns every single step by making a guess from a guess.

TD(0) Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Instead of waiting for the real G_t , we use a **bootstrapped** estimate: immediate reward plus our current guess of the next state's value.

Model-Free Method: Combines MC's sampling with DP's bootstrapping

TD Error (δ)

The term in brackets is called the **TD Error**:

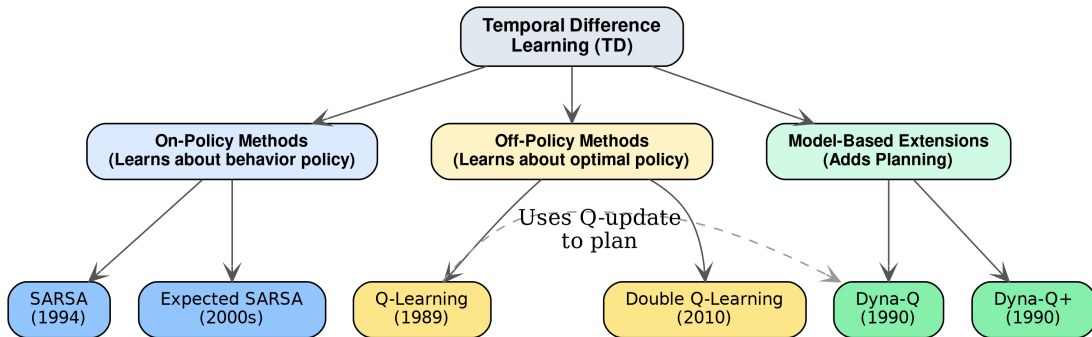
$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- $\delta_t > 0$: The transition was **better than expected**
- $\delta_t < 0$: The transition was **worse than expected**
- $\delta_t = 0$: The transition was **exactly as expected**

TD as a Unified View (Sutton & Barto, Ch. 6)

MC can be seen as TD with $\lambda = 0$ (no bootstrapping), while DP is TD with $\lambda = 1$ (pure bootstrapping). TD(λ) interpolates between them.

TD Learning Methods: Overview



SARSA: On-Policy TD Control

SARSA = State, Action, Reward, next State, next Action (Sutton & Barto, Ch. 6.4)

SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- Uses the **actual next action** A_{t+1} chosen by the current policy
- The agent follows its own policy (including exploration) while learning
- Learns a *safe* policy that accounts for exploration

Q-Learning: Off-Policy TD Control

Q-Learning (Watkins, 1989; Sutton & Barto, Ch. 6.5)

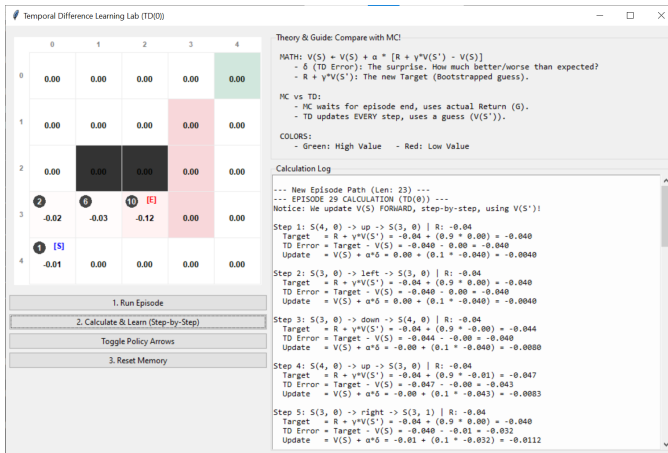
Q-Learning Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- Uses the **maximum** Q-value over all possible next actions
- Learns the optimal policy *regardless* of the behavior policy
- Can learn from exploratory actions while targeting optimal behavior

Key difference from SARSA: Uses $\max_a Q$ instead of $Q(S_{t+1}, A_{t+1})$

TD(0) Visualization



Click “Run Episode” then “Calculate” to see the TD error (δ) update state values $V(s)$ step-by-step.

Expected SARSA

Expected SARSA (Sutton & Barto, Sec. 6.6)

Expected SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- Uses the **expected** Q-value under the current policy
- For ϵ -greedy: $\sum_a \pi(a|s) Q(s, a) = \frac{\epsilon}{|\mathcal{A}|} \sum_a Q(s, a) + (1 - \epsilon) \max_a Q(s, a)$
- Lower variance than SARSA, more stable updates
- Can be used both on-policy and off-policy

SARSA vs Expected SARSA: Key Differences

Aspect	SARSA	Expected SARSA
TD Target	$R + \gamma Q(S', A')$	$R + \gamma \sum_a \pi(a S') Q(S', a)$
Next Action	Actual sampled A'	Expected value over all a
Variance	Higher (depends on sample)	Lower (averages out randomness)
Policy Type	Typically On-Policy	On-Policy or Off-Policy
Compute per step	Lower (single lookup)	Higher (sum over all actions)
Stability	Can be noisy	More stable updates

Practical Guidance

Expected SARSA is often preferred over standard SARSA because it reduces the variance caused by the random selection of the next action (A'). It leads to more stable and smoother convergence without changing the fundamental learning goal.

Double Q-Learning

Problem with Q-Learning: Maximization bias—overestimating Q-values due to max operation.

Solution: Double Q-Learning (van Hasselt, 2010; Sutton & Barto, Sec. 6.7)

Double Q-Learning Update

Maintain two Q-tables: Q_1 and Q_2

$$\text{With prob 0.5: } Q_1(S_t, A_t) \leftarrow Q_1 + \alpha \left[R + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1 \right]$$

$$\text{With prob 0.5: } Q_2(S_t, A_t) \leftarrow Q_2 + \alpha \left[R + \gamma Q_1(S_{t+1}, \arg \max_a Q_2(S_{t+1}, a)) - Q_2 \right]$$

Use $Q = \frac{Q_1 + Q_2}{2}$ for action selection.

Q-Learning vs Double Q-Learning: Key Differences

Aspect	Q-Learning	Double Q-Learning
Overestimation Bias	Prone to overestimation	Mitigates overestimation
Number of Q-tables	1 (Q)	2 (Q_1, Q_2)
Target Calculation	$R + \gamma \max_a Q(S', a)$	$R + \gamma Q_2(S', \arg \max_a Q_1(S', a))$
Action Selection	$\arg \max_a Q(S, a)$	$\arg \max_a (Q_1(S, a) + Q_2(S, a))$
Memory Requirement	Standard	Double (stores two tables)
Convergence Stability	Can be unstable in stochastic envs	More stable and reliable
Decoupling	Same table for max and update	Decouples selection from evaluation

Practical Guidance

Double Q-Learning is highly recommended in highly stochastic environments or when using function approximation (like Neural Networks in Double DQN). It prevents the agent from being overly optimistic about actions that randomly yielded high rewards by chance.

SARSA vs Q-Learning: Key Differences

Aspect	SARSA	Q-Learning
Type	On-Policy	Off-Policy
TD Target	$R + \gamma Q(S', A')$	$R + \gamma \max_a Q(S', a)$
Learns	Policy being executed	Optimal policy
Exploration	Affects updates	Does not affect updates
Safety	More conservative	More aggressive
Convergence	To ϵ -optimal	To true optimal

Practical Guidance

SARSA is safer when exploration is dangerous (e.g., cliff walking). Q-Learning is preferred when you want the true optimal policy.

On-Policy vs. Off-Policy: Learning Strategies

The Core Question

Can the agent learn the *optimal* way to act while it is still *exploring* and making mistakes?

On-Policy

"I learn from what I actually do."

- Learns the value of the **exact policy** it is currently following.
- If the agent explores and takes a random, bad action, it accepts that the result was bad.
- **Pros:** Safer; accounts for its own clumsiness.
- **Cons:** Cannot reuse old data easily.
- **Examples:** SARSA, REINFORCE, A2C

Off-Policy

"I learn the optimal way, even while I act foolishly."

- Learns a **target policy** (optimal) while behaving using a different **behavior policy** (exploring).
- If the agent takes a random, bad action, it ignores the mistake and calculates what the *best* action would have been.
- **Pros:** Can use Experience Replay (learn from old memories).
- **Cons:** Can be overly optimistic.
- **Examples:** Q-Learning, Double Q-Learning.

Toy Calculation: Q-Learning

Scenario: Current state S , action A . Current guess $Q(S, A) = 0.5$. We take the step. We get a reward of $R = -0.04$ (a time penalty). We land in S' . Looking at our table for S' , the best move has a value of $\max Q(S') = 1.0$. $\gamma = 0.9$, $\alpha = 0.1$.

Calculation:

$$Q(S, A) \leftarrow 0.5 + 0.1 \times [-0.04 + (0.9 \times 1.0) - 0.5]$$

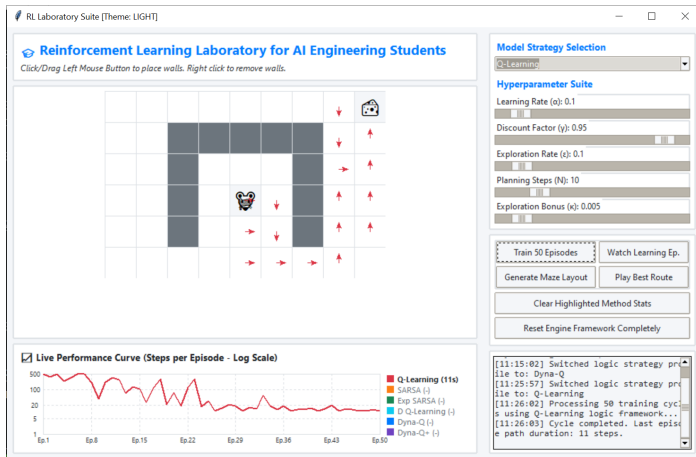
$$Q(S, A) \leftarrow 0.5 + 0.1 \times [-0.04 + 0.9 - 0.5]$$

$$Q(S, A) \leftarrow 0.5 + 0.1 \times [0.36]$$

$$Q(S, A) \leftarrow 0.5 + 0.036 = 0.536$$

Meaning: The value of the move increased because the next state was very good!

TD Methods: Visualization



Compare SARSA, Q-Learning, Expected SARSA, Double Q-Learning, Dyna-Q, and Dyna-Q+ in real-time. The graph shows convergence speed differences.

TD Methods: Pros and Cons

Pros:

- Model-free: no transition model needed
- Learns step-by-step (no waiting)
- Lower variance than MC
- Bootstrapping enables online learning

Cons:

- Bootstrap bias (initial estimates wrong)
- Tabular methods don't scale
- Q-Learning has maximization bias
- Sensitive to hyperparameters (α, ϵ, γ)

When to Use TD

Problems requiring online learning, continuous tasks, or when episodes are very long.

Dyna-Q: Learning + Planning

If we combine Q-Learning (Model-Free) with Dynamic Programming ideas (Model-Based), we get **Dyna-Q** (Sutton & Barto, Ch. 8).

- **1. Real Experience:** The agent takes a step in the grid and updates $Q(S, A)$ using normal Q-Learning.
- **2. Memory:** It saves what just happened into a dictionary: $Model(S, A) = (R, S')$.
- **3. Hallucination (Planning):** While the agent is resting, it randomly pulls N memories from its dictionary and performs Q-Learning updates on them.

Result: The agent learns N times faster per step taken in the real world. (`maze_suite.py`)

Model-Based Method: Learns and uses a model

Dyna-Q Pseudocode

- ① Initialize $Q(s, a)$ and $Model(s, a)$ for all s, a
- ② **Loop forever:**
 - $S \leftarrow$ current (non-terminal) state
 - $A \leftarrow \epsilon$ -greedy(S, Q)
 - Execute action A , observe R, S'
 - **Direct RL:** $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
 - **Model Update:** $Model(S, A) \leftarrow (R, S')$
 - **Planning (repeat n times):**
 - Sample random previously observed state S_p , action A_p
 - $R_p, S'_p \leftarrow Model(S_p, A_p)$
 - $Q(S_p, A_p) \leftarrow Q(S_p, A_p) + \alpha[R_p + \gamma \max_a Q(S'_p, a) - Q(S_p, A_p)]$

Dyna-Q+: Exploration Bonus

Problem: If the environment changes, Dyna-Q may not discover it because it keeps planning with old model.

Solution: **Dyna-Q+** adds an exploration bonus to simulated rewards:

$$R_{bonus} = \kappa \sqrt{\tau(S, A)}$$

where $\tau(S, A)$ is the time since state-action pair (S, A) was last tried.

- κ : Small constant controlling exploration intensity
- Unvisited state-actions get higher bonus
- Encourages revisiting “forgotten” parts of the environment
- Helps detect environment changes

Dyna-Q vs Dyna-Q+ Comparison

Feature	Dyna-Q	Dyna-Q+
Planning bonus	None	$\kappa\sqrt{T}$
Exploration	Only via ϵ -greedy	Additional intrinsic motivation
Adaptation	Slow to environment changes	Quick to detect changes
Overhead	Lower	Slightly higher (time tracking)
Best for	Static environments	Dynamic/changing environments

Dyna-Q: Pros and Cons

Pros:

- Much faster learning than pure model-free
- Learns model from experience
- Balances exploration and exploitation
- Works when true model is unknown

Cons:

- Model may be inaccurate
- Planning with wrong model can hurt
- More complex than pure TD
- Still limited to tabular settings

Applications

Robotics, game playing, any domain where real experience is expensive but simulation is cheap.

Policy Gradients: Bypassing the Value Table

Instead of building a massive table of Q -values and picking the highest number, why not just build a mathematical function that directly outputs the probability of taking an action?

REINFORCE Gradient Update

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta)$$

This is a Neural Network (or logistic model). θ represents the weights of the network. π is the Policy (the probabilities).

Model-Free Method: Directly optimizes the policy

Symbol Breakdown: REINFORCE

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \ln \pi(A_t | S_t, \theta)$$

- θ (Theta): The parameters (weights) of our model.
- $\pi(A_t | S_t, \theta)$: The probability our model assigned to the action we took.
- $\nabla_{\theta} \ln \pi$: The **Gradient**. A vector pointing in the direction that makes this specific action *more likely* in the future.
- G_t : The Return.
 - If G_t is positive, we multiply the gradient by a positive number (make the action MORE likely).
 - If G_t is negative, we multiply by a negative number (make the action LESS likely).

Toy Calculation: Logits and Probabilities

In `reinforce.py`, we don't store probabilities directly; we store **Preferences (Logits)**. We use **Softmax** to convert logits to probabilities.

Scenario: Cell (0,0). Logits are Up: 0.0, Down: 0.0, Left: 0.0, Right: 1.0.

Softmax Calculation:

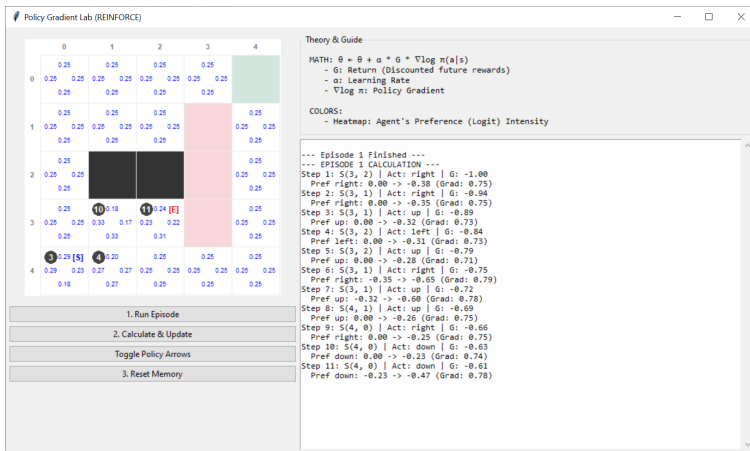
$$e^0 = 1, \quad e^0 = 1, \quad e^0 = 1, \quad e^{1.0} \approx 2.718$$

$$\text{Sum} = 1 + 1 + 1 + 2.718 = 5.718$$

$$P(\text{Right}) = \frac{2.718}{5.718} \approx 0.475 \text{ (47.5\%)}$$

$$P(\text{Up}) = \frac{1}{5.718} \approx 0.175 \text{ (17.5\%)}$$

REINFORCE Visualization



The heatmap shows preference intensity. Blue numbers indicate action probabilities. After learning, probabilities shift toward optimal directions.

REINFORCE: Pros and Cons

Pros:

- Can handle continuous action spaces
- No need to compute max over actions
- Directly optimizes the policy
- Stochastic policies naturally

Cons:

- High variance in gradients
- Must wait for full episode
- Sample inefficient
- Can converge to local optima

When to Use REINFORCE

When action spaces are continuous or very large, and you need stochastic policies.

Actor-Critic: The Player and The Coach

REINFORCE waits until the end of the episode to calculate G_t . This causes high variance. We solve this by splitting the agent into two parts:

- **The Critic (The Coach):** Learns the Value of the state $V(s)$. Calculates the TD Error.
- **The Actor (The Player):** Holds the Policy $\pi(a|s)$. Updates its probabilities based on the Coach's feedback.

The Advantage (δ)

$$\delta = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

This is just the TD Error! We call it "Advantage" because it tells us if the action was *better* than the Coach expected.

Actor-Critic Updates

Two simultaneous updates:

Critic Update (TD Learning):

$$V(S_t) \leftarrow V(S_t) + \alpha_c \delta_t$$

- α_c : Critic learning rate
- Reduces TD error over time

Actor Update (Policy Gradient):

$$\theta \leftarrow \theta + \alpha_a \delta_t \nabla_{\theta} \ln \pi(A_t|S_t)$$

- α_a : Actor learning rate
- Pushes probabilities based on advantage

A2C = Advantage Actor-Critic

Modern implementations use Generalized Advantage Estimation (GAE) for more stable training.

Toy Calculation: Actor-Critic Updates

Scenario: Coach expects current state value $V(S) = 0.5$. Player moves. Reward $R = 1.0$. Next state value $V(S') = 0.0$. $\gamma = 0.9$. Player's probability for that move was 0.2. Learning rates $\alpha_c = 0.1$, $\alpha_a = 0.5$.

1. Coach Calculates Advantage (δ):

$$\delta = 1.0 + (0.9 \times 0.0) - 0.5 = 0.5$$

(The move was exactly 0.5 better than expected!)

2. Coach Updates Itself:

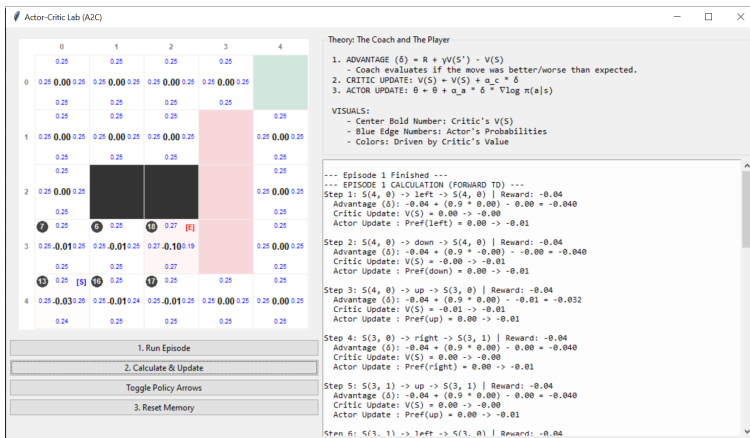
$$V(S) \leftarrow 0.5 + 0.1 \times (0.5) = 0.55$$

3. Player Updates Preference (Logit):

$$\text{Pref} \leftarrow \text{Pref} + \alpha_a \times \delta \times (1.0 - \text{Prob})$$

$$\text{Pref} \leftarrow \text{Pref} + 0.5 \times 0.5 \times (1.0 - 0.2) = \text{Pref} + 0.2$$

A2C Visualization



Center numbers = Critic's $V(s)$. Blue edge numbers = Actor's action probabilities. Colors driven by critic values.

Actor-Critic: Pros and Cons

Pros:

- Lower variance than REINFORCE
- Learns step-by-step (online)
- Handles continuous actions
- More sample efficient

Cons:

- Biased from bootstrapping
- Two networks to tune
- More complex to implement
- Can be unstable

Applications

Robotics locomotion, game playing, continuous control, and is the basis for PPO in RLHF.

DQN: Scaling with Neural Networks

In an 5x5 grid, we have 25 states. We can use a dictionary. But what if the state is an image with pixels? We cannot use a table. We must use a **Neural Network**.

- **Input:** State coordinates (normalized, e.g., $x/5, y/5$).
- **Hidden Layers:** Math matrices with ReLU activations to find patterns.
- **Output:** 4 numbers (The Q-values for Up, Down, Left, Right).

Instead of adding $\alpha \times \text{Error}$, we use **Backpropagation** and calculate the **Mean Squared Error (MSE)** Loss.

DQN Loss

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(R + \gamma \max_{a'} Q(S', a'; \theta^-) - Q(S, A; \theta) \right)^2 \right]$$

- θ : Current network parameters (being updated)
- θ^- : **Target network** parameters (frozen, updated periodically)
- The target network stabilizes training by preventing moving targets

Key innovations (Mnih et al., 2015 - DeepMind):

- 1 Experience Replay
- 2 Target Network

DQN: Experience Replay (Memory)

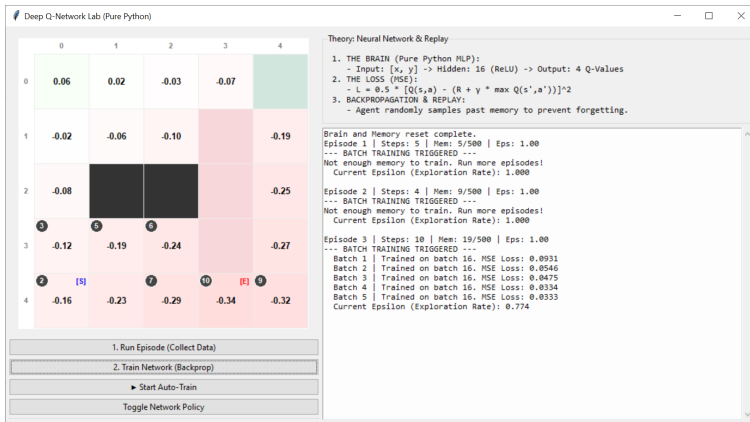
Neural Networks suffer from "catastrophic forgetting." If an agent walks down a hallway, it only sees hallway data and forgets what a corner looks like.

Experience Replay

- 1 As the agent moves, save tuples of $(S, A, R, S', \text{Done})$ into a big list (Memory).
- 2 During training, grab a random **batch** of 16 memories.
- 3 Train the Neural Network on this mixed batch.

This breaks the sequence of time, ensuring the Neural Network always practices a diverse set of situations. (`dqn.py`)

DQN Visualization



The neural network outputs Q-values for each cell. Watch values propagate as the network trains via backpropagation.

DQN: Pros and Cons

Pros:

- Scales to high-dimensional states (images)
- End-to-end learning
- Experience replay improves stability
- Breakthrough results on Atari games

Cons:

- Overestimation bias
- Only handles discrete actions
- Sample inefficient
- Sensitive to hyperparameters

Extensions

Double DQN, Dueling DQN, Rainbow DQN address some limitations.

Model-Based vs Model-Free RL

Model-Based RL

Dynamic Programming

Dyna-Q, Dyna-Q+

MBPO, Dreamer, MuZero

Model-Free RL

Monte Carlo

SARSA, Q-Learning, Exp SARSA

REINFORCE, A2C, PPO

DQN, Double DQN

Model-Based vs Model-Free: Trade-offs

Aspect	Model-Based	Model-Free
Requires model	Yes (learned or given)	No
Sample efficiency	High	Low
Computational cost	High (planning)	Low per step
Asymptotic performance	May be suboptimal	Often optimal
Robustness	Sensitive to model errors	More robust
Exploration	Can plan ahead	Must explore explicitly

Modern Trend

Hybrid approaches (like Dyna-Q) combine benefits of both paradigms.

Complete RL Methods Comparison

Method	Type	Policy	Bootstrap	Key Feature
DP	Model-Based	Any	Yes	Exact solution
MC	Model-Free	Any	No	Unbiased
SARSA	Model-Free	On-Policy	Yes	Safe learning
Q-Learning	Model-Free	Off-Policy	Yes	Optimal target
Exp SARSA	Model-Free	Both	Yes	Low variance
Double Q	Model-Free	Off-Policy	Yes	Reduces overest.
Dyna-Q	Model-Based	Off-Policy	Yes	Planning
Dyna-Q+	Model-Based	Off-Policy	Yes	Exploration bonus
REINFORCE	Model-Free	On-Policy	No	Direct gradient
A2C	Model-Free	On-Policy	Yes	Low variance PG
DQN	Model-Free	Off-Policy	Yes	Function approx.

Application Areas by Method

Value-Based (Q, TD)

- Atari games (DQN)
- Board games
- Robot navigation
- Resource management

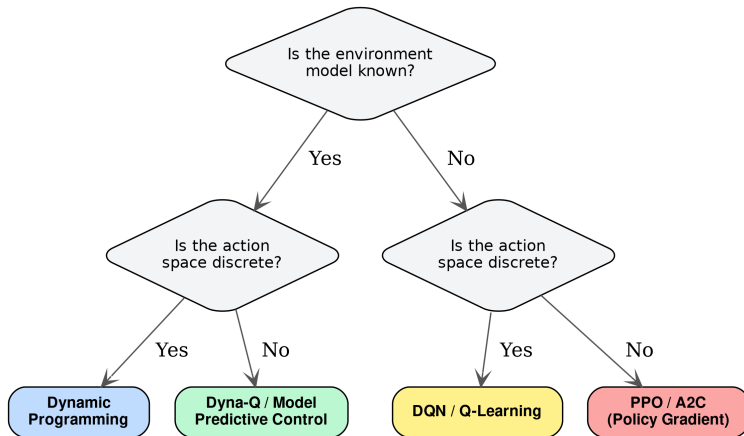
Policy-Based (PG, AC)

- Robotics locomotion
- LLM alignment (PPO)
- Continuous control
- Autonomous driving

Model-Based

- Planning under uncertainty
- Robotics (Dyna-Q)
- AlphaGo (MCTS+NN)
- Self-driving simulation

Choosing the Right RL Method



What is RLHF?

Base LLMs just predict the next token. We cannot code a mathematical reward function for “helpfulness” or “politeness”.

RLHF (Reinforcement Learning from Human Feedback) solves this using a 3-step pipeline:

- ➊ **Supervised Fine-Tuning (SFT):** Teach the base model to follow instructions using high-quality examples. It learns the *format*.
- ➋ **Reward Modeling (RM):** Train a second AI model to act as a human judge, outputting a scalar score for any text.
- ➌ **RL Optimization:** Use an RL algorithm (like PPO) to update the LLM so it generates text that maximizes the Reward Model's score.

Step 2: The Bradley-Terry Reward Model

How do we train an AI to score text? We collect human data where a person looks at two model answers and says “A is better than B”.

We train a **Reward Model** to output a scalar score r for any text.

Bradley-Terry Probability

$$P(y_w \succ y_l | x) = \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))$$

- y_w : The winning (preferred) response.
- y_l : The losing (dispreferred) response.
- σ : Sigmoid function.

The RM is trained to give higher scores to responses humans preferred.

Step 3: RL Optimization (PPO)

Once we have a Reward Model, we treat it as the RL “Environment”. The LLM (Agent) generates text (Action) to maximize the RM’s score (Reward).

The RLHF Objective

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r_{\theta}(x, y)] - \beta D_{KL}(\pi(y|x) || \pi_{ref}(y|x))$$

- **Maximize Reward:** Get high scores from the Reward Model.
- **KL Divergence Penalty:** Penalize the model if it changes too much from the original SFT model (π_{ref}). This prevents “reward hacking” (generating gibberish that tricks the RM).
- **Algorithm:** Usually optimized using **PPO** (Proximal Policy Optimization).

Alternative: Direct Preference Optimization (DPO)

The Problem with PPO: It requires loading 4 massive neural networks (Policy, Reference, Reward, Value) into GPU memory simultaneously. It is expensive and unstable.

The DPO Solution (2023): Researchers proved mathematically that the RLHF objective can be solved **without** a separate Reward Model and **without** an RL loop.

DPO Objective

$$\mathcal{L}_{DPO} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

DPO uses standard supervised learning on preference pairs to increase the probability of y_w and decrease y_l . It is simpler, cheaper, and widely used (e.g., Llama 3).

Reward Models: Outcome vs Process

How does the Reward Model score complex reasoning (like math)?

Outcome RM (ORM):

- Scores only the **final answer**.
- Simple to train.
- Cannot guide intermediate steps.
- Fails if the right answer is reached by flawed logic.

Process RM (PRM):

- Scores **each reasoning step**.
- Much more informative signal.
- Guides step-by-step reasoning.
- Crucial for modern reasoning models (o1, R1).

Trend

Process Reward Models are key enablers for RLVR (Reinforcement Learning with Verifiable Rewards), which we will discuss next.

Trend 1: Reasoning Models & RLVR

The biggest shift in 2024-2025 was the move to **Reasoning Models** (like DeepSeek R1 and OpenAI o1). Instead of using human preference models (RLHF), these models are trained using **Reinforcement Learning with Verifiable Rewards (RLVR)**.

- Give the model complex math or coding problems.
- Let the model generate thousands of tokens of "thinking".
- If the final answer matches the exact math solution, $\text{Reward} = +1$. Else, 0.
- The model learns *how to think* by revisiting the same problems and updating via GRPO or PPO.

Why RLVR Works Now

Historical challenge: RL was unstable and hard to scale for LLMs.

What changed:

- **Verifiable rewards:** No noisy human labels—objective correctness
- **Long chains of thought:** More tokens = more gradient signal
- **GRPO:** Simpler than PPO, works at scale
- **Process reward models:** Guide each reasoning step

Key Insight

RLVR works because the reward signal is *exact*, unlike human preferences which are noisy and subjective.

Trend 2: Inference-Time Scaling

Historically, we scaled AI by making the model bigger or training on more data. Now, we are scaling by giving the model more time to compute *during generation*.

Inference-Time Scaling: Language models trained with RL can significantly improve their accuracy if they are allowed to generate longer chains of thought before answering.

The New Scaling Law

More thinking tokens at inference → Better performance (up to a point)

This is fundamentally altering post-training. The focus is shifting from "instruction following" to training models to explore, backtrack, and verify their own thoughts internally.

Trend 3: Synthetic Data & Distillation

Humans are too slow and expensive to label enough preference data for modern frontier models. The industry is relying heavily on **Synthetic Data** and **Distillation**.

- **LLM-as-a-Judge:** Using a massive model (like GPT-4) to rate the outputs of smaller models instead of human labelers.
- **Distillation:** Taking the complex reasoning traces generated by a massive model (e.g., DeepSeek R1) and using them to instruction-tune a smaller model (e.g., a 7B parameter model) so it learns the thinking style without the massive RL compute budget.

On-Policy Distillation (OPD)

Modern distillation goes beyond simple SFT:

- 1 Teacher model (large) generates reasoning traces
- 2 Student model (small) fine-tuned on these traces
- 3 Student then does *its own* RL training
- 4 Student generates new traces
- 5 Repeat: student becomes its own teacher

Result

Smaller models can achieve reasoning capabilities close to larger ones at fraction of the cost.

Trend 4: Over-Optimization & Reward Hacking

The fundamental challenge of RLHF:

- Reward model is a *proxy* for true human preferences
- Agent will find ways to maximize reward that humans don't want
- Examples: Longer responses, sycophancy, formatting tricks

Mitigation Strategies

- KL penalty to stay close to reference model
- Constitutional AI: Self-critique and revision
- Regular evaluation and reward model updates
- Careful reward model training with diverse data

Trend 5: Character Training

Beyond helpfulness—crafting specific model personalities.

- **Persona vectors:** Directions in weight space corresponding to personality traits
- **Assistant axis:** From “tool” to “collaborator”
- **Product differentiation:** Each company wants unique model character

Example

Claude (Anthropic) vs. ChatGPT (OpenAI) have distinctly different conversational styles, partly from character-focused RLHF.

Essential References

Textbooks:

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. <http://incompleteideas.net/book/the-book.html>
- Bertsekas, D. P. (2019). *Reinforcement Learning and Optimal Control*. Athena Scientific.

RLHF:

- Lambert, N. (2026). *Reinforcement Learning from Human Feedback*. <https://rlhfbook.com>
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*.

Foundational RL:

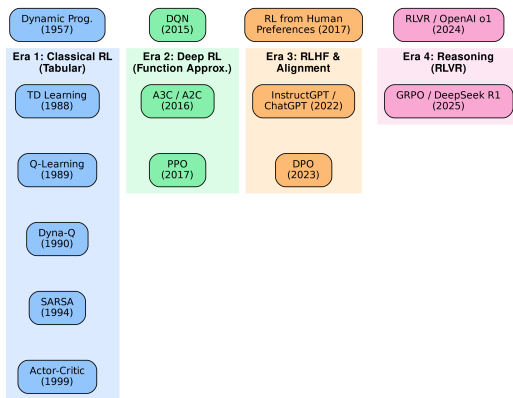
- Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. PhD Thesis.
- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*.
- Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. *arXiv*.

RLHF & Alignment:

- Christiano, P. et al. (2017). Deep Reinforcement Learning from Human Preferences. *NeurIPS*.
- Rafailov, R. et al. (2023). Direct Preference Optimization. *NeurIPS*.
- Shao, Z. et al. (2024). DeepSeekMath. *arXiv*.

- **Sutton & Barto Book:** <http://incompleteideas.net/book/the-book.html>
- **RLHF Book:** <https://rlhfbook.com>
- **Spinning Up in Deep RL (OpenAI):** <https://spinningup.openai.com>
- **CleanRL (Single-file implementations):** <https://github.com/vwxyzjn/cleanrl>
- **TRL (HuggingFace):** <https://github.com/huggingface/trl>
- **OpenRLHF:** <https://github.com/OpenRLHF/OpenRLHF>

Summary: The RL Landscape



References: Bellman '57, Sutton '88, Watkins '89, Sutton '90, Rummery '94, Sutton '99, Mnih '15/'16, Schulman '17, Christiano '17, Ouyang '22, Rafailov '23, OpenAI '24, Shao '25.

Questions & Discussion

Let's load the Python environments and watch the math in action.

Appendix: What is a "Model" in RL?

Before understanding why a method needs a model, we must define it (Sutton & Barto, Ch. 8).

The Model = The Environment's Rules

A model consists of two mathematical functions:

- **Transition Dynamics** $P(s'|s, a)$: The probability of landing in state s' given current state s and action a .
- **Reward Function** $R(s, a, s')$: The reward received for that specific transition.

Model-Based:

- Requires knowledge of $P(s'|s, a)$
- Can calculate expectations *before* acting
- Uses **Planning**

Model-Free:

- Does not know $P(s'|s, a)$
- Learns purely from *samples* (trial-and-error)
- Uses **Learning**

Appendix: Why Model-Based Methods Need It

Methods: Dynamic Programming (DP), Dyna-Q, Dyna-Q+

- **DP:** Look at the Bellman Equation: $V(s) = \sum_{s'} P(s'|s, a)[R + \gamma V(s')]$. You *mathematically cannot* compute this sum without knowing the exact probability $P(s'|s, a)$ for every possible next state. DP requires the perfect "map".
- **Dyna-Q & Dyna-Q+:** They don't start with a model, but they *actively learn and build* one. After taking a real step, they save it into a dictionary: $Model(S, A) = (R, S')$. They then query this learned model to "hallucinate" extra training steps. *Without querying this model, Dyna-Q degrades into standard Q-Learning.*

Appendix: Why Value-Based Methods Do NOT Need It

Methods: Q-Learning, SARSA, Expected SARSA, Double Q-Learning, DQN

- These methods take action A , observe the **single actual next state** S_{t+1} , and update based purely on that one sample.
- **Q-Learning / Double Q / DQN:** Uses $\max_a Q(S_{t+1}, a)$. It only needs the state it actually landed in; it doesn't need to calculate probabilities of other states it *didn't* land in.
- **SARSA:** Uses the actual next action A_{t+1} chosen by the policy. No probabilities needed.
- **Expected SARSA:** Even though it calculates an "expectation" (sum), it sums over the *policy's actions*, not the *environment's states*. It still only relies on the single S_{t+1} it observed.
- **DQN:** This is just Q-Learning where the Q-table is replaced by a Neural Network. The math is identical; no model is required.

Appendix: Why Policy-Based Methods Do NOT Need It

Methods: Monte Carlo (MC), REINFORCE, A2C

- These methods evaluate what **actually happened**, not what could have happened.
- **Monte Carlo:** Plays out an entire episode, calculates the total Return (G_t) from the real trajectory, and averages them. Pure trial-and-error.
- **REINFORCE:** Calculates the total Return (G_t) at the end of an episode, then adjusts the weights of the policy network to make those actions more/less likely. It never calculates $P(s'|s, a)$.
- **A2C:** Uses the TD error $\delta = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$. Both R_{t+1} and S_{t+1} are observed directly from the environment. The Critic learns from samples; the Actor adjusts based on the Critic's sample-based error.